```
            return Optional.empty();    #B
        }

        var expiry =
claims.getExpirationTime().toInstant();    #C
        var subject = claims.getSubject();    #C
        var token = new Token(expiry, subject);    #C

        String scope;    #D
        try {     #D
            scope = claims.getStringClaim("scope");    #D
        } catch (ParseException e) {    #D
            scope = String.join(" ",     #D
                claims.getStringListClaim("scope"));    #D
        }    #D
        token.attributes.put("scope", scope);     #D
        return Optional.of(token);

    } catch (ParseException | BadJOSEException |
JOSEException e) {
        return Optional.empty();
    }
}
```

#A Verify the signature first.
#B Ensure the issuer and audience have expected values.
#C Extract the JWT subject and expiry time.
#D The scope may be either a string or an array of strings.

CHOOSING A SIGNATURE ALGORITHM

The JWS standard that JWT uses for signatures supports many different public key signature algorithms, summarized in table 7.2. Because public key signature algorithms are expensive and usually limited in the amount of data that